

SkillSwap — Software Requirements Specification (SRS)

1. Project Title

SkillSwap — Intelligent Peer-to-Peer Learning Platform with Optimized Matching, Scheduling, and Reputation Algorithms

2. Introduction

2.1 Purpose

SkillSwap is a distributed web and mobile-based learning ecosystem that enables users to exchange skills with one another using a credit-based system instead of direct monetary transactions.

The system allows users to act as both learners and mentors. Users can teach skills to earn credits and use those credits to learn new skills from other mentors.

The project integrates algorithmic techniques from Design and Analysis of Algorithms (DAA) including:

- Mentor-Learner Matching Algorithms
- Session Scheduling Optimization
- Graph-Based Reputation Ranking
- Recommendation Systems
- Credit Optimization

The same intelligent backend powers both:

- Flutter mobile applications
- Laravel web application

This ensures consistency across platforms.

2.2 Scope

The system will provide:

- User authentication and profile management
- Skill publishing and browsing
- Intelligent mentor recommendation

- Session scheduling and optimization
- Real-time messaging
- Credit wallet system
- Graph-based reputation ranking
- Session reviews and ratings
- Recommendation engine
- Multi-platform access (Web + Mobile)

The project focuses on solving real-world optimization problems related to matching, scheduling, ranking, and recommendation.

3. Project Objectives

The objectives of the system are:

1. Create a peer-to-peer learning ecosystem.
 2. Implement intelligent mentor matching algorithms.
 3. Optimize session scheduling.
 4. Build a scalable distributed system.
 5. Apply DAA concepts in real-world software engineering.
 6. Provide consistent functionality across web and mobile platforms.
 7. Implement graph-based reputation and ranking.
 8. Build reusable backend APIs.
-

4. Technologies Used

4.1 Frontend Technologies

Web Application

- HTML
- CSS
- JavaScript
- jQuery

Mobile Application

- Flutter
 - Dart
-

4.2 Backend Technologies

- PHP
 - Laravel Framework
 - REST APIs
-

4.3 Database

- PostgreSQL
-

4.4 Development Tools

- VS Code
 - Postman
 - Git & GitHub
 - Composer
 - Flutter SDK
-

5. System Architecture

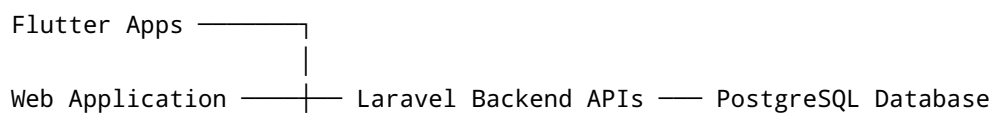
5.1 High-Level Architecture

The system consists of:

1. Laravel Backend Server
2. PostgreSQL Database
3. Flutter Mobile Applications
4. Laravel Web Application
5. Algorithmic Service Layer

All intelligent algorithms are implemented in the backend and shared across all clients.

5.2 Architecture Flow



6. User Roles

6.1 Learner

A learner can:

- Search mentors
- Request sessions
- Chat with mentors
- Attend sessions
- Rate mentors
- Spend credits

6.2 Mentor

A mentor can:

- Publish skills
- Accept session requests
- Teach sessions
- Earn credits
- Set availability
- Receive ratings

6.3 Admin

Admin can:

- Monitor users
- Manage reports
- Handle abuse reports
- View analytics
- Manage skills

7. Functional Requirements

7.1 Authentication Module

Features

- User registration
- User login
- Password reset
- Email verification
- Session management
- Logout

Inputs

- Name
- Email
- Password

Outputs

- Authenticated user session
-

7.2 User Profile Module

Features

- Create profile
- Update profile
- Upload profile picture
- Add bio
- Add interests

Inputs

- Profile details
- Image upload

Outputs

- Updated user profile
-

7.3 Skill Marketplace Module

Features

- Add skills
- Browse skills
- Search skills
- Filter mentors
- View mentor profiles

Inputs

- Skill title
- Description
- Experience level

Outputs

- Skill listings
 - Mentor recommendations
-

7.4 Mentor-Learner Matching Module

Features

- Intelligent mentor ranking
- Top-K mentor recommendation
- Skill relevance scoring
- Availability matching
- Experience-based filtering

Inputs

- User skill request
- Availability
- Ratings

Outputs

- Ranked mentor list

Algorithms Used

- Priority Queue
- Heap-based Ranking
- Weighted Scoring

Complexity

- $O(n \log k)$
-

7.5 Session Scheduling Module

Features

- Session request creation
- Conflict-free scheduling
- Session approval/rejection
- Availability management
- Auto time-slot optimization

Inputs

- Requested time
- Mentor availability

Outputs

- Optimized session schedule

Algorithms Used

- Greedy Interval Scheduling
- Dynamic Programming

Complexity

- $O(n \log n)$
-

7.6 Messaging Module

Features

- Real-time chat
- Send messages
- Receive messages
- Conversation history

Inputs

- Message text

Outputs

- Chat communication
-

7.7 Credit Wallet Module

Features

- Credit earning
- Credit spending
- Wallet balance
- Transaction history

Inputs

- Session completion

Outputs

- Updated wallet balance
-

7.8 Reputation System Module

Features

- Mentor ratings
- Learner feedback
- Trust score generation

- Reputation ranking

Inputs

- Ratings
- Reviews

Outputs

- Reputation score

Algorithms Used

- Graph-Based Ranking
- PageRank-inspired Algorithm

Complexity

- $O(V + E)$
-

7.9 Recommendation Module

Features

- Skill recommendations
- Mentor suggestions
- Personalized learning paths

Inputs

- User interaction history
- Ratings
- Popular skills

Outputs

- Recommended content
-

8. Non-Functional Requirements

8.1 Performance

- API response time under 2 seconds
 - Efficient query optimization
 - Fast mentor matching
-

8.2 Scalability

The system should support:

- Thousands of users
 - Concurrent sessions
 - Multiple API requests
-

8.3 Security

- Password hashing
 - Token-based authentication
 - SQL injection prevention
 - Input validation
 - Secure APIs
-

8.4 Availability

- 24/7 accessibility
 - Reliable database connection
-

8.5 Usability

- Responsive UI
 - Simple navigation
 - Cross-platform support
-

9. Database Design

Main Tables

User Tables

- users
- profiles

Skill Tables

- skills
- user_skills

Session Tables

- sessions
- session_requests
- availability_slots

Financial Tables

- wallets
- transactions

Communication Tables

- messages
- conversations

Reputation Tables

- reviews
- trust_scores

10. Data Structures Used

Heap / Priority Queue

Used for:

- Mentor ranking
 - Top-K recommendation
-

Graph

Used for:

- Reputation system
 - Trust relationships
-

Dynamic Arrays / Lists

Used for:

- Session management
 - Skill listings
-

Hash Maps

Used for:

- Fast lookup
 - User indexing
-

11. Algorithmic Components

11.1 Mentor Matching Algorithm

Objective

Find the best mentor for a learner.

Factors

- Rating
- Experience
- Availability
- Skill relevance
- Response rate

Algorithm

Weighted scoring + priority queue.

11.2 Scheduling Optimization

Objective

Maximize sessions while avoiding conflicts.

Algorithms

- Greedy scheduling
- Dynamic programming

11.3 Reputation Ranking

Objective

Compute fair mentor trust score.

Algorithm

PageRank-style graph ranking.

11.4 Recommendation Engine

Objective

Recommend skills and mentors.

Methods

- Weighted scoring
- Collaborative filtering

12. API Requirements

Authentication APIs

- Register
 - Login
 - Logout
 - Reset password
-

Skill APIs

- Add skill
 - Get skills
 - Search skills
-

Session APIs

- Create request
 - Accept session
 - Reject session
 - Fetch sessions
-

Messaging APIs

- Send message
 - Fetch messages
-

Wallet APIs

- Get balance
 - Fetch transactions
-

13. User Interface Requirements

Web Application

- Responsive design

- Dashboard layout
 - Tables and filters
 - Real-time updates
-

Mobile Application

- Android support
 - Clean navigation
 - Real-time interactions
 - Optimized mobile UI
-

14. Testing Requirements

Functional Testing

- Authentication testing
 - Session testing
 - Matching testing
 - Scheduling testing
-

Performance Testing

- API load testing
 - Query optimization testing
-

Security Testing

- SQL injection testing
 - Authentication testing
-

15. Future Enhancements

- AI chatbot integration
 - Video call integration
 - Machine learning recommendations
 - Voice-based learning assistance
 - Multi-language support
-

16. Expected Outcomes

The system is expected to:

- Improve peer learning accessibility
 - Efficiently match mentors and learners
 - Optimize scheduling automatically
 - Provide fair reputation rankings
 - Demonstrate practical applications of DAA concepts
 - Support both mobile and web platforms consistently
-

17. Conclusion

SkillSwap is an intelligent distributed learning platform that combines modern web and mobile technologies with advanced algorithmic concepts. The project demonstrates practical implementation of Design and Analysis of Algorithms concepts including graph algorithms, scheduling optimization, ranking systems, and recommendation engines.

The system is designed to provide a scalable, efficient, and user-friendly learning ecosystem powered by intelligent computational methods.